

Luxury Touch Massage - Developer Guide

Welcome to the codebase for the **Luxury Touch Massage** management system. This guide is intended for developers taking over maintenance, updates, or deployment of the system. Even if you have limited experience with these specific technologies, this guide maps out the core architecture and setup process.

1. System Architecture Overview

The project is split into three main components functioning together, powered by Firebase.

A. The Database (Firebase Firestore)

The entire application runs on Firebase Firestore. There is no traditional relational database or backend server running Node.js handling the API routes anymore (the legacy `server.js` was replaced by direct Firebase integrations).

- **Collection `site_content` -> Document `landing_page`:** Stores a JSON object containing the site's branding, contact information, services, and add-ons.
- **Collection `bookings`:** Stores individual documents for every customer booking request.

B. The Frontend / Client Site (`message_admin_app/client_site/`)

This is the customer-facing booking website.

- **Stack:** Pure HTML, Vanilla CSS (`styles.css`), and Vanilla JavaScript (`script.js`).
- **How it works:** It uses the Firebase Web SDK (`firebase-app-compat.js` and `firebase-firestore-compat.js`) to read directly from the `site_content` collection in real-time, and write directly into the `bookings` collection.
- **Note:** The legacy `script_nodesserver.js` interact with the old node server and is no longer the main file.

C. The Admin App (`message_admin_app/`)

This is the business owner's control panel.

- **Stack:** Flutter (Dart) targeting Android, iOS, and Web.
- **How it works:** Uses `cloud_firestore` and `firebase_core` to read from and write to both the `bookings` collection (to approve/reschedule) and the `site_content` collection (to update services and branding).

D. Legacy Components (`junko-san_project/` root)

At the root of the project, you will find:

- `server.js` (Express Node server), `data.json`, and `bookings.json`. These are **deprecated**. The system used to rely on this local Node server.
- `migrate.js`: A Node.js script that was used to push the local `data.json` into the Firebase `site_content` collection during the initial transition.

2. Prerequisites & Setup

If you are setting up this project on a new machine or a new Firebase instance, follow these steps.

Step 2.1: Tooling

- Install **Node.js** (if you need to run `migrate.js` or the legacy server).
- Install **Flutter SDK** (v3.10+ recommended) and **Dart**.
- Set up an emulator (Android Studio / Xcode) or web browser for Flutter testing.

Step 2.2: Firebase Configuration

Currently, the website and the Flutter app are pointing to the `junko-san-05zvr7` Firebase project. If you are taking ownership of the system, you should create your own Firebase project.

1. Create a project at [Firebase Console](#).
2. Enable **Firestore Database** in Test Mode (or define strict rules for production).
3. **Important:** Obtain your new Firebase Web Config keys.

Step 2.3: Update Keys in the Web Client

Open `message_admin_app/client_site/index.html`. Around line 60, find the `firebaseConfig` object and replace it with your newly generated keys:

```
const firebaseConfig = {  
  apiKey: "YOUR_API_KEY",  
  authDomain: "YOUR_AUTH_DOMAIN",  
  projectId: "YOUR_PROJECT_ID",  
  //...  
};
```

Step 2.4: Update Keys in the Flutter App

The Flutter app uses `firebase_options.dart` (located in `message_admin_app/lib/firebase_options.dart`). To reconfigure the Flutter app for your new project:

1. Install the Firebase CLI: `npm install -g firebase-tools`
2. Run `firebase login`
3. Run `dart pub global activate flutterfire_cli`
4. CD into `message_admin_app` and run: `flutterfire configure`
5. Select your newly created Firebase project. This will regenerate `firebase_options.dart`.

Step 2.5: Seeding the Database

If your new Firebase Firestore is empty, the website and app will crash or show no data.

1. Ensure your new Firebase project generates a `serviceAccountKey.json`.

2. Place `serviceAccountKey.json` in the root `junko-san_project/` folder.
3. Run the migration script to push the initial dataset:

```
node migrate.js
```

3. Running & Developing Locally

Running the Web Client

You can run the web client using any simple HTTP server.

1. Navigate to the client directory: `cd message_admin_app/client_site/`
2. Run a python server (`python -m http.server 8000`) or Node (`npm serve`).
3. Open `http://localhost:8000` in your browser.

Running the Flutter Admin App

1. Open a terminal and navigate to: `cd message_admin_app`
2. Fetch dependencies: `flutter pub get`
3. Run the app on an available device/emulator: `flutter run` *Tip: Use `flutter devices` to see available targets (e.g., chrome, android, ios).*

4. Deployment Guide

Deploying the Web Client (Firebase Hosting)

Firebase Hosting is the easiest way to deploy the static HTML/JS/CSS client site.

1. Navigate to `message_admin_app/client_site/`
2. Run `firebase init hosting`
3. Select your project. Set the public directory to the current folder (`.`). Do not configure as a single-page app.
4. Run `firebase deploy --only hosting`
5. The CLI will provide you with a live URL (e.g., `https://your-project.web.app`).

Deploying the Flutter Admin App

- **Android:** Run `flutter build apk --release` or `flutter build appbundle` (for Google Play).
- **iOS:** Run `flutter build ipa` (requires a Mac with Xcode and Apple Developer account).
- **Web:** Run `flutter build web`. You can then host the contents of `build/web/` onto Firebase Hosting or any standard web server.

5. Security Note regarding Rules

Currently, the client side integrates directly with Firestore. Ensure your `firestore.rules` are configured properly for production so that regular users can only `create` bookings, while only authenticated Admins

can **update** bookings or the **site_content**.

Example Basic Rule:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Only allow public reading of site_content, NO writing.
    match /site_content/{document} {
      allow read: if true;
      allow write: if false; // Add auth check here for admins
    }
    // Allow public creating of bookings, reading/updating only for admins.
    match /bookings/{document} {
      allow create: if true;
      allow read, update, delete: if false; // Add auth check here for
admins
    }
  }
}
```

Note: Make sure to implement Firebase Authentication if you use strict rules.